Resolución de problemas aplicando el modelo relacional, cardinalidad y normalización GA6-220501096-AA1-EV01

Presentado por:

Luis Alejandro Muñoz Sierra

Presentado a:

Naín Zúñiga

Servicio Nacional de Aprendizaje – SENA

Programa de Formación:

Análisis y desarrollo de software

Ficha: 2977356

Medellín, 10 de Octubre de 2025

Resolución de problemas aplicando el modelo relacional, cardinalidad y normalización.

Software de Gestión de Maquinaria Pesada

Introducción

En el desarrollo de cualquier sistema de software, la base de datos es un componente fundamental que garantiza la integridad, persistencia y consistencia de la información. Para el proyecto "MaquiMu", un software orientado a la gestión de alquiler de maquinaria pesada, el diseño de un modelo de datos robusto es crítico para el éxito operativo.

Este documento presenta el diseño del modelo relacional para el sistema MaquiMu. A partir de los requerimientos del software, se detallan las entidades identificadas, las relaciones y la cardinalidad que existen entre ellas, y el análisis del proceso de normalización aplicado para asegurar un diseño eficiente, escalable y libre de redundancias.

Objetivo General

Diseñar el modelo relacional de la base de datos para el software "MaquiMu", asegurando que cumpla con los estándares de la industria y sirva como una base sólida para la construcción de la aplicación.

Objetivos Específicos

- Identificar y presentar las entidades (tablas) que componen el modelo de datos del sistema.
- Definir y documentar la cardinalidad de las relaciones entre las diferentes entidades.
- Analizar y aplicar las reglas de normalización (hasta la Tercera Forma Normal - 3FN) para garantizar la integridad de los datos y eliminar redundancias.

Análisis del Modelo a Desarrollar

Según la documentación que hemos realizado previamente ya el análisis existía y se basa en los requerimientos consolidados del proyecto anteriormente, que incluyen la gestión de inventario, clientes, alquileres y finanzas.

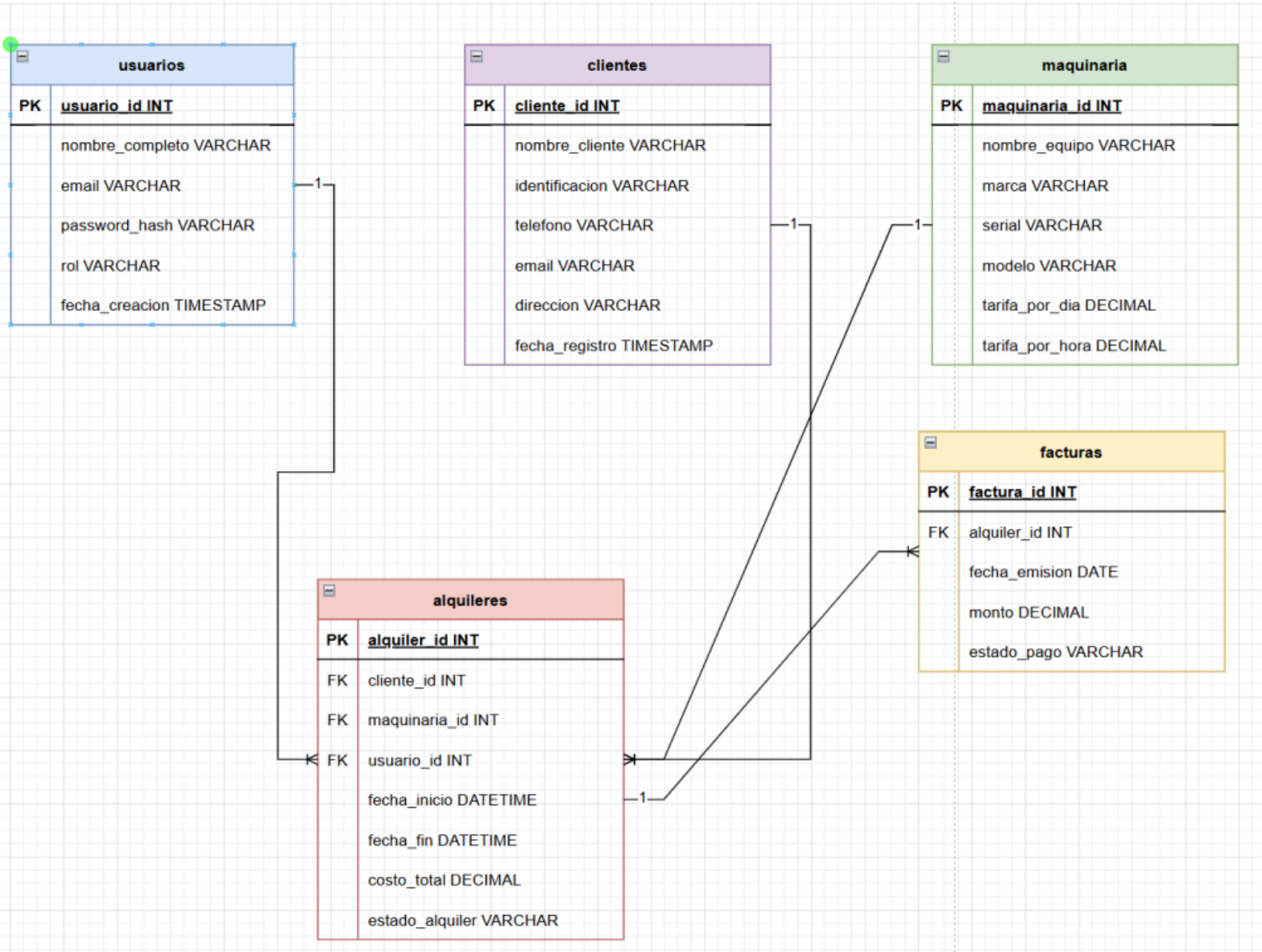
1. Identificación de Entidades (Modelo Relacional)

Basado en el análisis de los requerimientos, se han identificado y presentado a lo largo del proyecto cinco (5) entidades principales que se traducen en las siguientes tablas para el modelo relacional:

1. **`usuarios`**: Almacena las cuentas de los empleados (Administradores, Agentes) que operan el sistema.
2. **`clientes`**: Contiene la información de las personas o empresas que solicitan los alquileres.
3. **`maquinaria`**: Funciona como el inventario central, registrando cada equipo disponible para alquilar.

4. **`alquileres`**: Es la tabla transaccional principal que registra cada contrato de alquiler, vinculando un cliente, una máquina y un agente.
5. **`facturas`**: Almacena la información financiera (montos, estados de pago) asociada a cada alquiler.

Modelo Relacional (Diccionario de Datos):



2. Establecimiento de Cardinalidad

La cardinalidad define cómo se relacionan las entidades entre sí. En este modelo, las relaciones son:

`Clientes` (1) a (N) `Alquileres`: Un cliente puede tener muchos alquileres a lo largo del tiempo, pero un alquiler pertenece a un solo cliente. Se implementa con la llave foránea `cliente_id` en la tabla `alquileres`.

`Usuarios` (1) a (N) `Alquileres`: Un usuario (agente) puede gestionar muchos alquileres, pero un alquiler es gestionado por un solo usuario. Se implementa con la llave foránea `usuario_id` en la tabla `alquileres`.

`Maquinaria` (1) a (N) `Alquileres`: Una máquina puede estar asociada a muchos alquileres (en diferentes momentos), pero un registro de alquiler específico se refiere a una sola máquina. Se implementa con la llave foránea `maquinaria_id` en la tabla `alquileres`.

`Alquileres` (1) a (N) `Facturas`: Un alquiler puede generar una o más facturas (por ejemplo, pagos parciales o recurrentes), pero una factura está asociada a un solo alquiler. Se implementa con la llave foránea `alquiler_id` en la tabla `facturas`.

3. Aplicación de Normalización

Para garantizar la integridad de los datos, evitar la redundancia y prevenir anomalías de actualización, el modelo relacional se ha diseñado aplicando las formas normales:

Primera Forma Normal (1FN):

- **Concepto:** Todos los atributos (columnas) deben ser atómicos; no deben existir grupos de repetición ni valores múltiples en una sola celda.

- **Aplicación:** El modelo cumple con la 1FN. Cada columna almacena un solo valor. Por ejemplo, en la tabla **`clientes`**, los campos **`telefono`** y **`email`** son atómicos. No existe un campo "teléfonos" que almacene una lista. De igual manera, en **`maquinaria`**, las tarifas **`tarifa_por_dia`** y **`tarifa_por_hora`** están en columnas separadas y atómicas.

Segunda Forma Normal (2FN):

- **Concepto:** El modelo debe estar en 1FN y todos los atributos no clave deben depender funcionalmente de la *totalidad* de la clave primaria. Esta regla es relevante principalmente para tablas con claves primarias compuestas.
- **Aplicación:** El modelo cumple con la 2FN. Todas las entidades (**`usuarios`**, **`clientes`**, **`maquinaria`**, **`alquileres`**, **`facturas`**) utilizan una clave primaria simple (un solo campo, ej: **`cliente_id`**). Al no haber claves primarias compuestas, no existen dependencias parciales. Todos los atributos no clave (como **`nombre_cliente`** o **`direccion`** en la tabla **`clientes`**) dependen completamente de la clave primaria (**`cliente_id`**).

Tercera Forma Normal (3FN):

- **Concepto:** El modelo debe estar en 2FN y no deben existir dependencias transitivas (un atributo no clave no debe depender de otro atributo no clave).
- **Aplicación:** El modelo cumple con la 3FN. La información se ha segregado en entidades lógicas para evitar dependencias transitivas.
- **Ejemplo:** Los detalles del cliente (como **`nombre_cliente`** y **`direccion`**) se almacenan en la tabla **`clientes`**. Si almacenáramos la **`direccion`** del cliente en la tabla **`alquileres`**, se crearía una dependencia transitiva (Alquiler -> Cliente -> Dirección). Al separarlos, la **`direccion`** depende directamente de **`cliente_id`** (su clave primaria), y la tabla **`alquileres`** solo almacena la referencia **`cliente_id`**.

- De igual forma, los detalles de la máquina (como **`marca`** y **`serial`**) están en la tabla **`maquinaria`**, evitando que estos datos se repitan redundantemente en cada registro de la tabla **`alquileres`**.

Conclusiones

El diseño del modelo relacional para el proyecto "MaquiMu" se ha completado exitosamente, cumpliendo con todos los objetivos planteados. A través de este ejercicio, se han consolidado las bases técnicas para la persistencia de datos del software.

Se identificaron y definieron las cinco entidades centrales del negocio: usuarios, clientes, maquinaria, alquileres y facturas. Para estas entidades, se establecieron relaciones claras de cardinalidad, predominantemente de uno a muchos (1..N), lo que refleja adecuadamente el flujo operativo (un cliente tiene muchos alquileres, un alquiler tiene varias facturas, etc.).

El análisis de normalización confirma que el modelo propuesto cumple satisfactoriamente hasta la Tercera Forma Normal (3FN). Al aplicar la 1FN, 2FN y 3FN, se ha garantizado un diseño que elimina la redundancia de datos, previene anomalías de inserción, actualización y borrado, y asegura la integridad referencial.

Este modelo relacional, validado y normalizado, constituye una base sólida y eficiente sobre la cual el equipo de desarrollo podrá construir los componentes de la capa de persistencia y la lógica de negocio del sistema "MaquiMu".

HISTORIAL DE MODIFICACIONES

Versión	Naturaleza de cambio	fecha de aprobación	Fecha validación
1.0	Presentación inicial	2025-10-12	2025-10-12